Proiect de curs TSS

Tema proiectului de curs va fi aleasă din lista de mai jos. Fiecare temă din listă poate fi aleasă de cel mult 2 echipe. Dimensiunea (maximă a) echipei pentru fiecare tema este specificata mai jos.

Aceste proiecte vor fi prezentate la curs în perioada 27.10.2026 - 22.12.2026 (mai puțin 01.12.2026, sărbătoare publică) conform programării stabilite, astfel încât numărul maxim de studenți care prezintă în cadrul unui curs să fie 10.

Prezentarea proiectului de curs va fi sub forma de slide-uri, insotite de demo-uri. La prezentare este necesara prezenta intregii echipe, fiecare student descriind principala sa contributie la proiect. Timpul alocat fiecarui student este de 10 minute.

La finalul proiectului, **echipa va declara procentul de contribuție al fiecărui membru**. Procentele declarate trebuie să însumeze 100%.

Utilizarea instrumentelor bazate pe inteligență artificială

Pentru elaborarea proiectului, echipele **pot utiliza instrumente bazate pe inteligență artificială**, precum ChatGPT, Claude, Gemini sau alte instrumente similare.

Pentru fiecare **utilizare semnificativă a instrumentelor AI**, echipa va documenta:

- instrumentul AI utilizat;
- scopul utilizării;
- promptul sau descrierea solicitării adresate instrumentului AI;
- rezultatul obținut;
- modificările efectuate de membrii echipei asupra rezultatului;
- modul în care rezultatul a fost verificat și validat.

La prezentarea finală, echipa trebuie să poată **explica utilizările semnificative ale instrumentelor AI** și să demonstreze că rezultatele generate au fost **analizate, verificate și validate de membrii echipei** înainte de a fi utilizate în cadrul proiectului.

Teme proiecte curs

1. **FSM based testing**
   - Prezentati problematica testarii bazate pe FSM, tehnica W-method si ilustrati aplicarea acesteia pe un exemplu.
   - Scrieti un program care implementeaza W-method.

   Marime echipa: 2 studenti
   Bibliografie: [1,2]

2. **Search based testing pentru EFSM**
   - Prezentati metoda de generare de date de test prezentata in [3].
   - Scrieti un program care implementeaza metoda de generare de date de test prezentata in [3] si ilustrati functionarea acesteia pe un exemplu.

   Marime echipa: 2 studenti
   Bibliografie: [3]

3. **Automatic Evolutionary Test Suite Generation cu EvoSuite**
    - Prezentati utilitarul EvoSuite, care generează automat cazuri de testare cu aserțiuni pentru clase scris în cod Java folosind o abordare hibridă pentru generarea si optimizarea întregii suite de testare ce satisface un criteriu de acoperire.
    - Ilustrati capabilitatile utilitarului folosind unul sau mai multe exemple create de echipa.

    Marime echipa: 2 studenti
    Bibliografie: [4,5]

4. **Automatic Search-based Test Case Generation for Autonomous Systems cu AmbieGen**
    - Prezentati framework-ul AmbieGen, care generează automat cazuri de testare pentru sisteme autonome folosind o căutare evolutivă pentru a identifica cele mai critice scenarii pentru un sistem dat.
    - Ilustrati capabilitatile utilitarului folosind unul sau mai multe studii de caz create de echipa.

    Marime echipa: 2 studenti
    Bibliografie: [6,7]

5. **Model based testing cu JSXM**
    - Prezentati limbajul de modelare bazat pe stream X-machines precum si principalele facilitati oferite de utilitarul JSXM: animarea modelului, generarea testelor pe baza modelului de forma stream X-machine, implementarea testelor in JUnit.
    - Ilustrati aceste capabilitati folosind unul sau mai multe exemple create de echipa.

    Marime echipa: 3 studenti
    Bibliografie: [8-10]
    Observatie: pentru instalarea tool-ului se va consulta documentul „JSXM installation instructions".

6. **Model based testing for Cloud Services cu Broker@Cloud**
    - Prezentati limbajul de modelare bazat pe stream X-machines precum si principalele facilitati oferite de utilitarul Broker@Cloud verification and testing tool: verificare, test generation si test grounding.
    - Ilustrati aceste capabilitati folosind unul sau mai multe exemple create de echipa.

    Marime echipa: 2 studenti
    Bibliografie: [11,12]

7. **Model based testing cu GraphWalker**
    - Prezentati principalele facilitati oferite de utilitarul GraphWalker: generarea testelor din modelul masinilor cu stari finite (FSM) folosind algoritmi de traversare precum A* sau parcurgere random cu diverse criterii de acoperire (state, edge, requirement).
    - Ilustrati aceste capabilitati folosind unul sau mai multe exemple create de echipa.

    Marime echipa: 2 studenti
    Bibliografie: [13,14]

8. **Modelare si analiza cu Event-B, Rodin**
    - Prezentati capabilitatile de modelare si analiza ale limbajului Event-B: evenimente, rafinare, demonstrare automata a proprietatilor.
    - Ilustrati aceste capabilitati folosind unul sau mai multe exemple create de echipa.

    Marime echipa: 3 studenti

Bibliografie: [15,16]

9. Model checking cu NuSMV

- Prezentati pe scurt logica LTL si logica CTL.
- Ilustrati capabilitatile de model checking (verificare de proprietati LTL si CTL) folosind unul sau mai multe exemple create de echipa.

Marime echipa: 3 studenti
Bibliografie: [17,18]

10. Automatic test generation with Randoop

- Prezentați principalele facilitati oferite de utilitarul Randoop: generarea aleatorie de teste bazată pe feedback, crearea dinamică de secvențe de apeluri de metode cu intrări aleatorii și executarea acestora pentru a explora diferite căi de cod, a detecta erori și a asigura o acoperire cuprinzătoare a testelor pentru programele Java.
- Ilustrati aceste capabilitati folosind unul sau mai multe exemple create de echipa.

Marime echipa: 2 studenti
Bibliografie: [19,20]

11. Behavior Driven Development (BDD) with Cucumber

- Prezentati metodologia BDD și modul în care Cucumber ajută la scrierea testelor în Gherkin.
- Ilustrati aceasta metodologie folosind unul sau mai multe exemple create de echipa.

Marime echipa: 2 studenti
Bibliografie: [21,22]

12. Testarea bazată pe Contracte (Design by Contract)

- Prezentati tehnica „Design by Contract" și aplicarea acesteia în testarea sistemelor, inclusiv folosind icontract pentru Python.
- Ilustrati aceasta tehnica folosind unul sau mai multe exemple create de echipa.

Marime echipa: 2 studenti
Bibliografie: [23,24]

13. Property-Based Testing with Hypothesis (Python)

- Prezentati tehnica testării bazate pe proprietăți și utilizarea bibliotecii Hypothesis pentru generarea automată a cazurilor de test.
- Ilustrati aceasta tehnica folosind unul sau mai multe exemple create de echipa.

Marime echipa: 2 studenti
Bibliografie: [25-27]

14. Foundational Property-Based Testing in Haskell

- Prezentati tehnica Foundational Property-Based Testing si explicati diferentele fata de Property-Based Testing.
- Ilustrati aceasta tehnica folosind unul sau mai multe exemple create de echipa in Haskell.

Marime echipa: 3 studenti
Bibliografie: [28]

15. Contract Testing with Pact

- Prezentati tehnica testării contractuale pentru microservicii și utilizarea Pact pentru asigurarea compatibilității între client și server.
- Ilustrati aceasta tehnica folosind unul sau mai multe exemple create de echipa.

Marime echipa: 2 studenti
Bibliografie: [29,30]

16. Combinatorial / Pairwise Testing

- Prezentati tehnica de testare combinatoriala (pairwise/t-wise) si modul in care reduce numarul de cazuri de test pentru sisteme cu mai multi parametri configurabili.
- Ilustrati aceste capabilitati folosind un utilitar precum PICT sau ACTS pe unul sau mai multe exemple create de echipa.

Marime echipa: 2 studenti
Bibliografie: [31,32]

17. Symbolic Execution

- Prezentati tehnica de executie simbolica pentru generarea automata de teste (path constraints, SMT solving, path explosion).
- Ilustrati aceste capabilitati folosind un utilitar precum KLEE (C) sau Java PathFinder (Java) pe unul sau mai multe exemple create de echipa.

Marime echipa: 3 studenti
Bibliografie: [33,34]

18. Model-Based Testing cu UPPAAL

- Prezentati formalismul automatelor temporizate (timed automata) si principalele facilitati oferite de UPPAAL pentru modelare, verificare si generare de teste pentru sisteme cu constrangeri de timp real.
- Ilustrati aceste capabilitati folosind unul sau mai multe exemple create de echipa.

Marime echipa: 3 studenti
Bibliografie: [35,36]

19. Testarea API-urilor cu Schemathesis

- Prezentati tehnica de property-based fuzzing pentru API-uri REST pe baza specificatiilor OpenAPI/Swagger si modul in care Schemathesis genereaza automat cazuri de test din aceste specificatii.
- Ilustrati aceste capabilitati folosind unul sau mai multe exemple create de echipa.

Marime echipa: 2 studenti
Bibliografie: [37,38]

20. Testarea sistemelor de Machine Learning

- Prezentati provocarile specifice testarii sistemelor de machine learning (oracle problem, robustete, atacuri adversariale, validarea datelor) si cum difera de testarea software clasica.
- Ilustrati aceste capabilitati folosind un utilitar precum Adversarial Robustness Toolbox sau Deepchecks pe unul sau mai multe exemple create de echipa.

Marime echipa: 3 studenti
Bibliografie: [39,40,41]

Bibliografie

[1]. Aditya P. Mathur. Foundations of Software Testing, Pearson Education 2008. Chapter 6: Test Generation: FSM Models.
[2]. https://personal.utdallas.edu/~ewong/SE6367/03-Lecture/92-Test-Gen-based-on-FSM.pdf
[3]. Raluca Lefticaru, Florentin Ipate: Automatic State-Based Test Generation Using Genetic Algorithms, in SYNASC, 188-195 (2007). http://www.ifsoft.ro/~florentin.ipate/publications/SYNASC%202007%20Automatic%20State-Based%20Test%20Generation%20Using%20Genetic%20Algorithms.pdf
[4]. https://www.evosuite.org/
[5]. Gordon Fraser, Andrea Arcuri: EvoSuite: automatic test suite generation for object-oriented software. SIGSOFT FSE 2011: 416-419 https://www.researchgate.net/profile/Andrea-Arcuri-3/publication/221560749_EvoSuite_Automatic_test_suite_generation_for_object-oriented_software/links/595cd3e50f7e9b3aefaddbd0/EvoSuite-Automatic-test-suite-generation-for-object-oriented-software.pdf
[6]. https://arxiv.org/abs/2301.01234
[7]. https://github.com/swat-lab-optimization/AmbieGen-tool
[8]. Florentin Ipate, Mike Holcombe: Specification and testing using generalized machines: a presentation and a case study, Software Testing, Verification and Reliability, 8, 61-81 (1998) http://www.ifsoft.ro/~florentin.ipate/publications/STVR98%20-%20Specification%20and%20testing%20using%20generalised%20machines.pdf
[9]. http://www.jsxm.org/
[10]. Dimitris Dranidis, Konstantinos Bratanis, Florentin Ipate: JSXM: a tool for automated test generation, in SEFM, 352-366 (2012). http://www.ifsoft.ro/~florentin.ipate/publications/SEFM2012_CR.pdf
[11]. http://staffwww.dcs.shef.ac.uk/people/A.Simons/broker/
[12]. Lefticaru, R. and Simons, A.J.H. (2015) X-Machine Based Testing for Cloud Services. In: Ortiz, G. and Tran, C., (eds.) Advances in Service-Oriented and Cloud Computing. Workshops of ESOCC 2014, September 2-4, 2014, Manchester, UK. Lecture Notes in Computer Science, 508 . Springer , pp. 175-189. https://eprints.whiterose.ac.uk/98321/1/XMtestingforcloud.pdf
[13]. https://graphwalker.github.io/
[14]. Muhammad Nouman Zafar, Wasif Afzal, Eduard Enoiu, Athanasios Stratis, Aitor Arrieta, Goiuria Sagardui: Model-Based Testing in Practice: An Industrial Case Study using GraphWalker. ISEC 2021: 5:1-5:11 http://ebiltegia.mondragon.edu/xmlui/bitstream/handle/20.500.11984/5411/Model-Based%20Testing%20in%20Practice_An%20Industrial%20Case%20%20Study%20using%20GraphWalker.pdf?sequence=1&isAllowed=y
[15]. Jean-Raymond Abrial. Modeling in Event-B: System and Software Engineering.
[16]. http://www.event-b.org/
[17]. https://www.cs.utexas.edu/users/moore/acl2/seminar/2010.05-19-krug/slides.pdf
[18]. https://nusmv.fbk.eu/
[19]. https://homes.cs.washington.edu/~mernst/pubs/pacheco-randoop-oopsla2007.pdf
[20]. https://randoop.github.io/randoop/
[21]. https://cucumber.io/docs/guides/
[22]. https://knowledgecenter.ubt-uni.net/cgi/viewcontent.cgi?article=3575&context=conference
[23]. https://pypi.org/project/icontract/
[24]. https://pypi.org/project/design-by-contract/
[25]. https://hypothesis.readthedocs.io/en/latest/

[26]. https://arjancodes.com/blog/how-to-use-property-based-testing-in-python-with-hypothesis/
[27]. https://joss.theoj.org/papers/10.21105/joss.01891.pdf
[28]. https://inria.hal.science/hal-01162898/document
[29]. https://docs.pact.io/
[30]. https://pactflow.io/blog/what-is-contract-testing/
[31]. https://github.com/microsoft/pict
[32]. D. Richard Kuhn, Raghu N. Kacker, Yu Lei: Introduction to Combinatorial Testing. CRC Press (2013).
[33]. https://klee.github.io/
[34]. Cristian Cadar, Daniel Dunbar, Dawson Engler: KLEE: Unassisted and Automatic Generation of High-Coverage Tests for Complex Systems Programs. OSDI (2008).
[35]. https://uppaal.org/
[36]. Gerd Behrmann, Alexandre David, Kim G. Larsen: A Tutorial on Uppaal. SFM (2004).
[37]. https://schemathesis.readthedocs.io/
[38]. Zac Hatfield-Dodds, Dmitry Dygalo: Deriving Semantics-Aware Fuzzers from Web API Schemas. ICSE (2022).
[39]. https://github.com/Trusted-AI/adversarial-robustness-toolbox
[40]. https://deepchecks.com/
[41]. Jie M. Zhang, Mark Harman, Lei Ma, Yang Liu: Machine Learning Testing: Survey, Landscapes and Horizons. IEEE Transactions on Software Engineering, 48(1), 1-36 (2022).